

OAuth의 흐름

야 너두 소셜로그인 구현할 수 있어.

박상윤 (2025. 02. 13)

목차

1. OAuth(Open Authorization)란?

2. OAuth의 인증 과정

3. 따라해보기

3.1. 사전작업

3.2. Authorize url

3.3. Token API 호출

3.4. Userinfo API 호출

4. 각 서비스 제공자 별 특징

4.1. Kakao

4.2. Naver

4.3. Google

4.4. Apple(가장 싫어!!!!)

1. OAuth(Open Authorization)란?

개념

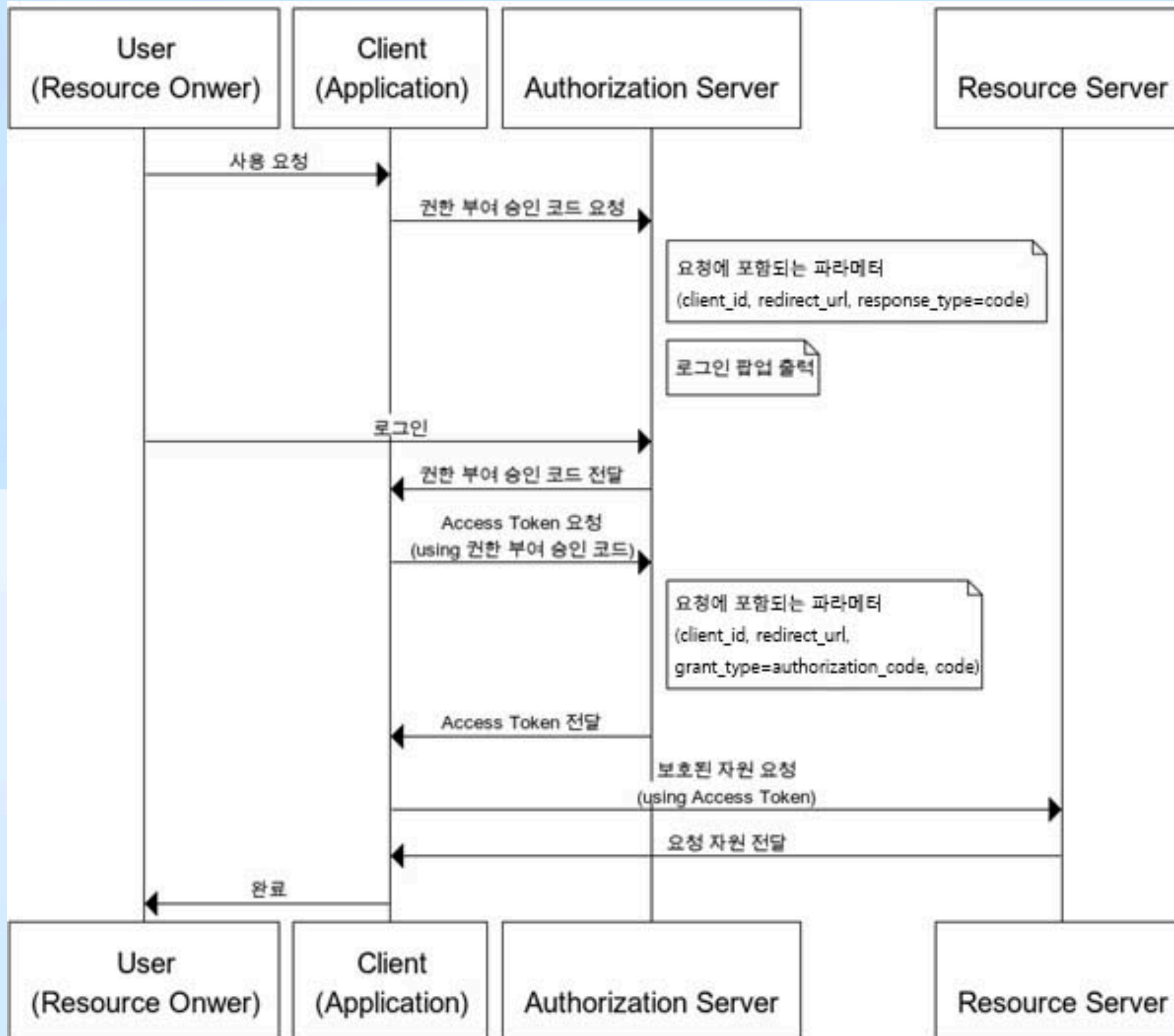
- 구글, 페이스북, 카카오와 같은 다양한 플랫폼의 특정한 사용자 데이터에 접근하기 위해 제 3자가 사용자의 접근 권한을 위임받을 수 있는 표준 프로토콜
- 특정 인가 흐름을 사용하면 클라이언트가 비밀번호를 제공하지 않고도 보호된 리소스에 액세스할 수 있음
- 클라이언트는 비밀번호 대신 리소스에 대한 액세스 권한을 부여할 해당 리소스의 소유자가 승인할 수 있는 액세스 토큰을 사용하게 된다

1. OAuth(Open Authorization)란?

주요 용어

- Resource Owner : 계정을 보유하고 있는 사용자
- Authorization Server : Resource Owner의 인가를 확인하고 Access Token을 발급하는 서버
- Resource Server : Access Token을 검증하고 보호된 자원을 제공하는 서버
- Access Token : Resource Server에게서 보호된 자원을 얻기위해 사용되는 만료기간이 있는 토큰(만료기간이 짧음)
- Refresh Token : Access Token이 만료되었을 때 이를 갱신하기 위해 사용하는 토큰 (Access Token보단 만료기간이 길다)

2. OAuth의 인증과정



3. 따라해보기

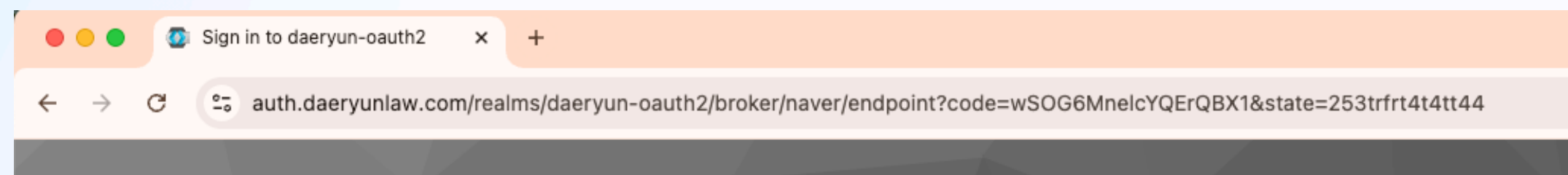
3-1. 사전작업

1. 어플리케이션 생성(그냥 생성하면 됨)
2. 로그인 사용설정(다른건 필요에 따라서 설정하면 되고 redirect url은 꼭 설정해줘야 한다)
3. Client id 와 Client secret은 미리 확보해둔다.

3. 따라해보기

3-2. Authorize url

1. 여기서는 api를 사용하지 않고 로그인 창을 사용함
2. 공통 파라미터(정확한 정보는 각 서비스 제공자 문서를 확인해야 한다)
 1. response_type : 응답형태, 로그인 할때는 일반적으로 code를 사용한다
 2. client_id : 인증 제공자에게 부여받은 id
 3. redirect_uri : 로그인 성공시 이동할 url(미리 등록되어 있어야 한다)
3. 선택적 파라미터(혹은 인증 제공자에서 별도로 요구하는 파라미터)
 1. scope : 제공받고자 하는 정보 범위(예: email, name 등)
 2. state : 임의의 문자열. csrf 방지용으로도 사용되기도 하고, 로그인 성공까지 가져가야 할 값들을 넣는 용도로 사용하기도 한다.



3. 따라해보기

3-3. Token API 호출

1. 로그인에 성공하면 미리 세팅한 redirect url로 이동한다.
2. 이때 제공받는 parameter가 code(추가로 state)이다. 이 파라미터를 이용해서 token을 가져온다.
3. api호출방식이며 method는 post, body type은 x-www-form-urlencoded이다.
4. 주요 파라미터
 - 4.1.grant_type : 인증방식(주로 authorization_code로 고정한다)
 - 4.2.client_id, client_secret : 인증 제공자에게 부여받은 id 및 보안키
 - 4.3.code : 로그인 성공시 받은 응답코드
 - 4.4.redirect_uri : 로그인 요청시 사용했던 등록된 redirect_uri
5. 성공하면 access_token과 refresh_token 등 인증에 필요한 token 목록을 받아온다.
6. 그렇다고 이것 직접 사용자 정보를 가져올 수 있지는 않다.(서비스 제공자 별로 살짝 차이는 있다)

3. 따라해보기

3-3. Token API 호출

1. 성공하면 아래와 같이 토큰을 받을 수 있다

POST <https://nid.naver.com/oauth2.0/token>

Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☒ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

Key	Value
☒ grant_type	authorization_code
☒ client_id	[REDACTED]
☒ client_secret	[REDACTED]
☒ code	[REDACTED]
☒ state	[REDACTED]

Body Cookies Headers (21) Test Results | ⌚

Pretty Raw Preview Visualize JSON

```
1 {
2   "access_token": "[REDACTED]",
3   "refresh_token": "[REDACTED]",
4   "token_type": "bearer",
5   "expires_in": "3600"
6 }
```

3. 따라해보기

3-4. Userinfo API 호출

1. 성공하면 아래와 같이 정보를 제공 받을 수 있다

The screenshot displays a Postman interface for a GET request to `https://openapi.naver.com/v1/nid/me`. The Headers tab is active, showing a list of headers with checkboxes. The Authorization header is set to `Bearer [redacted]`. The Body tab is also active, showing a JSON response in Pretty format. The response contains a `resultcode` of "00", a `message` of "success", and a `response` object with user details.

Key	Value	Description
☒ Postman-Token	<calculated when request is sent>	
☒ Host	<calculated when request is sent>	
☒ User-Agent	PostmanRuntime/7.43.0	
☒ Accept	*/*	
☒ Accept-Encoding	gzip, deflate, br	
☒ Connection	keep-alive	
☒ Authorization	Bearer [redacted]	

```
1 {
2   "resultcode": "00",
3   "message": "success",
4   "response": {
5     "id": [redacted],
6     "email": [redacted],
7     "mobile": [redacted],
8     "mobile_e164": [redacted],
9     "name": "[redacted]"
10  }
11 }
```

3. 따라해보기

3-4. Userinfo API 호출

1. 여기까지 왔다면 다 왔다
2. Token API에서 받은 `access_token` 을 이용해서 사용자 정보를 요청한다.
3. header 에 'bearer {access_token}' 으로 요청하면 된다.
4. 나머지는 받은 정보를 이용하면 된다.
5. 끝

4. 각 서비스 제공자 별 특징

4-1. Kakao

- 제공항목중에 sub가 없다. 필요하다면 수동으로 만들어주면 된다.
- 필요하다면 id값을 sub에 추가해준다.

4. 각 서비스 제공자 별 특징

4-1. Naver

- Id token을 제공해 주지 않는다.
- pc service url을 하나만 등록할 수 있다. referer를 체크하기 때문에 하나의 어플리케이션에 여러개의 서비스를 등록하려면 로그인 요청할때 referer url을 제거해서 보내야 한다.

• Java :

```
response.setHeader(s: "Referrer-Policy", s1: "no-referrer");  
response.sendRedirect(redirectUri);
```

• Next :

```
import { useRouter } from 'next/router';  
  
const redirectWithNoReferrer = (url) => {  
  const a = document.createElement('a');  
  a.href = url;  
  a.rel = 'noreferrer'; // Referrer 정보 제거  
  a.click();  
};  
  
const Component = () => {  
  const router = useRouter();  
  
  const handleRedirect = () => {  
    redirectWithNoReferrer('https://example.com');  
  };  
  
  return <button onClick={handleRedirect}>Go to External Site</button>;  
};
```

4. 각 서비스 제공자 별 특징

4-1. Google

- 크게 이슈는 없다

4. 각 서비스 제공자 별 특징

4-1. Apple

- 최초 로그인을 할때 이메일을 제공할지를 선택하는데 선택하지 않으면 임의의 id를 제공해 준다. (이후 변경 불가능함)
- email 을 받기위해 scope=email로 하려면 reponse_mode=form_post로 설정해야 하는데 이렇게 하면 redirect url 을 post 방식으로 이동하기 때문에 이에 맞춰서 처리해줘야한다.
- client secret을 직접 생성해야 하는데 이때 해당 서비스 key file을 이용해야하며 최장 6개월밖에 유지가 안된다(6개월에 한번씩 갱신해줘야 한다.)
- userinfo api가 없다. 필요한 정보는 id token을 복호화해서 처리하면 된다. access token은 서비스 해지할때만 사용한다.

QnA

감사합니다